Tutorial 5 - Digital Systems

1. Design an Excess-3 to Binary converter using AND gates, OR gates and inverters. Develop the appropriate truth tables and Boolean equations (for the outputs) in terms of minterms and don't cares (take unused cases as don't cares). Perform the appropriate simplification and draw the circuit.

   Note: Excess-3 (XS-3) code is a self-complementary binary-coded decimal (BCD) system in which each decimal digit (0-9) is represented by its binary equivalent plus 3. It is useful in error detection. (Since it is self-complementary, if a digit is incorrectly represented, the resulting code may not represent a valid Excess-3 representation.)

2. Design a 4-to-2 priority encoder with inputs D_0, D_1, D_2 and D_3 such that D_3 has the highest priority followed by D_2 and so on. Draw the appropriate truth table and develop the logic equations for the two outputs in terms of the inputs. You may ignore the third output (whose purpose is to resolve the ambiguity relating to all inputs being 0). Incorporate also the various don't care cases in your solution and perform simplification of the expressions for the outputs.

3. Consider the multiplexer shown in Figure 1, where I_0, I_1, I_2, I_3 are four data input lines that can be selected using the two address line combinations A_1A_0 which take values of $00, 01, 10$ and 11 respectively, and EN is the enable input (active high). Find the expression for f implemented by the circuit.

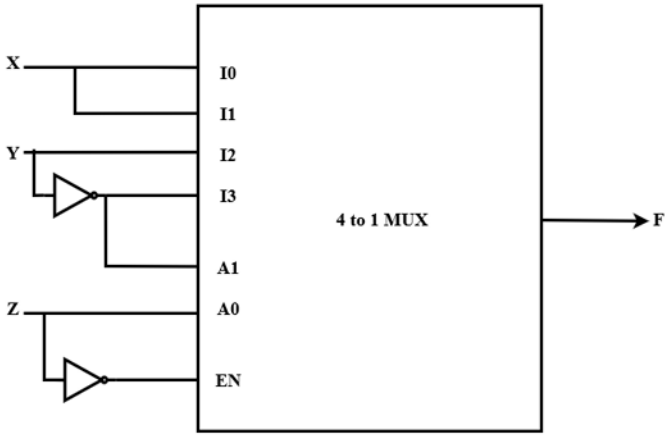


Figure 1: 4-to-1 Multiplexer (MUX)

4. Realize a 16-to-1 MUX using only 2-to-1 MUXes. Draw the circuit.

5. Consider the 3-input majority function. Design the logic circuit for this using (a) 8:1 Mux and (b) 4:1 Mux. Choose A(MSB),B and C as inputs.

6. We have already seen the role of Gray code in Karnaugh maps. In this question, we will look at a conversion task involving 4-bit binary and Gray codes. Consider Table 1

where the input (binary) is given by *abcd* while the corresponding output, denoted by $g_3g_2g_1g_0$, is in Gray code format. Implement the output g_1 using NOR gates and outputs g_2 and g_0 using NAND gates.

a	b	c	d	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Table 1: Truth Table for Inputs and Outputs

7. Implement the function $F(A, B, C, D) = \sum(1, 2, 3, 4, 7, 8, 9, 10, 12, 13, 15)$ using:

   (a) An 8:1 MUX and required logic gates.

   (b) A 4:1 MUX and required logic gates.

Draw the implementation tables for both the cases.

8. Construct the truth table and output expressions Y_0, Y_1, Y_2, Y_3 for the combinational circuit in Figure 2 with input I and s_1 (MSB), s_0 as the select lines. Comment on its operation by comparing it with a multiplexer.

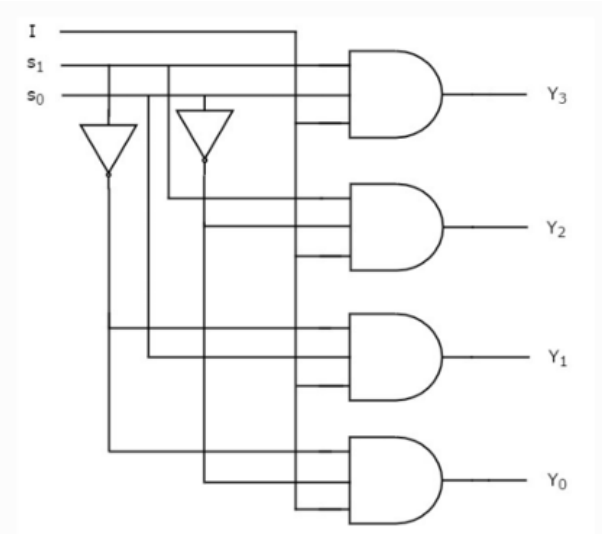
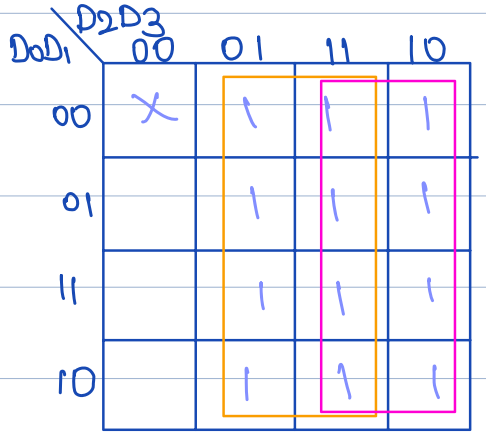


Figure 2: Circuit with logic gates

2. Design a 4-to-2 priority encoder with inputs D_0, D_1, D_2 and D_3 such that D_3 has the highest priority followed by D_2 and so on. Draw the appropriate truth table and develop the logic equations for the two outputs in terms of the inputs. You may ignore the third output (whose purpose is to resolve the ambiguity relating to all inputs being 0). Incorporate also the various don't care cases in your solution and perform simplification of the expressions for the outputs.

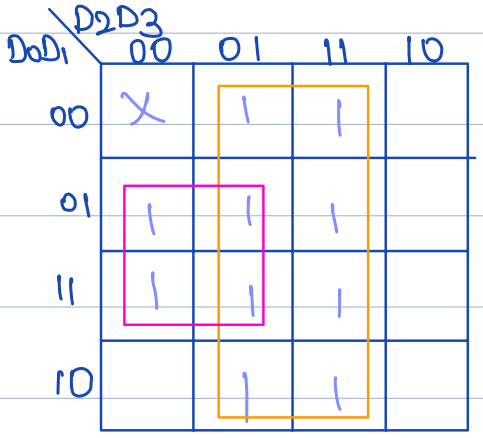
D_3	D_2	D_1	D_0	x	y	V
1	x	x	x	1	1	1
0	1	x	x	1	0	1
0	0	1	x	0	1	1
0	0	0	1	0	0	1
0	0	0	0	x	x	0

x:

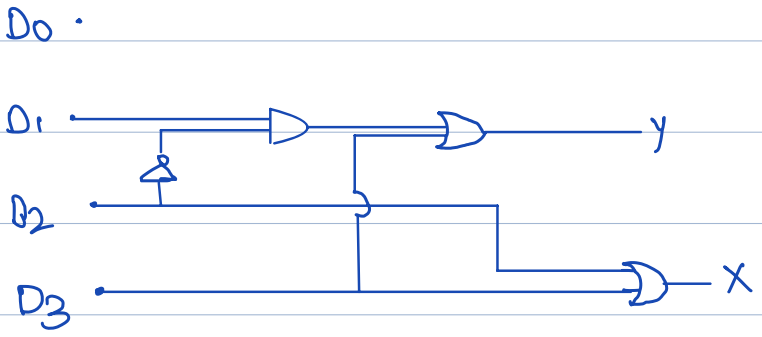


$x = D_2 + D_3$

y:



$y = D_3 + D_1 \overline{D_2}$



3. Consider the multiplexer shown in Figure 1, where I_0, I_1, I_2, I_3 are four data input lines that can be selected using the two address line combinations A_1A_0 which take values of $00, 01, 10$ and 11 respectively, and EN is the enable input (active high). Find the expression for f implemented by the circuit.

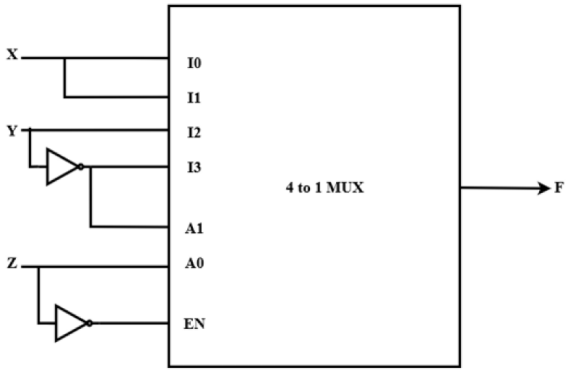


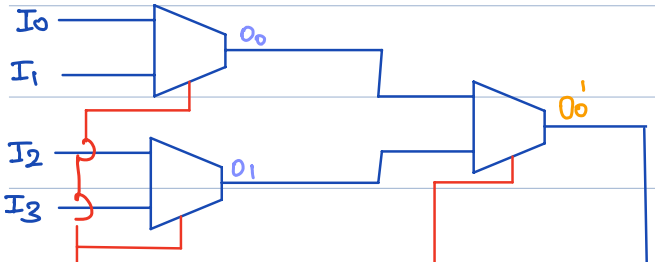
Figure 1: 4-to-1 Multiplexer (MUX)

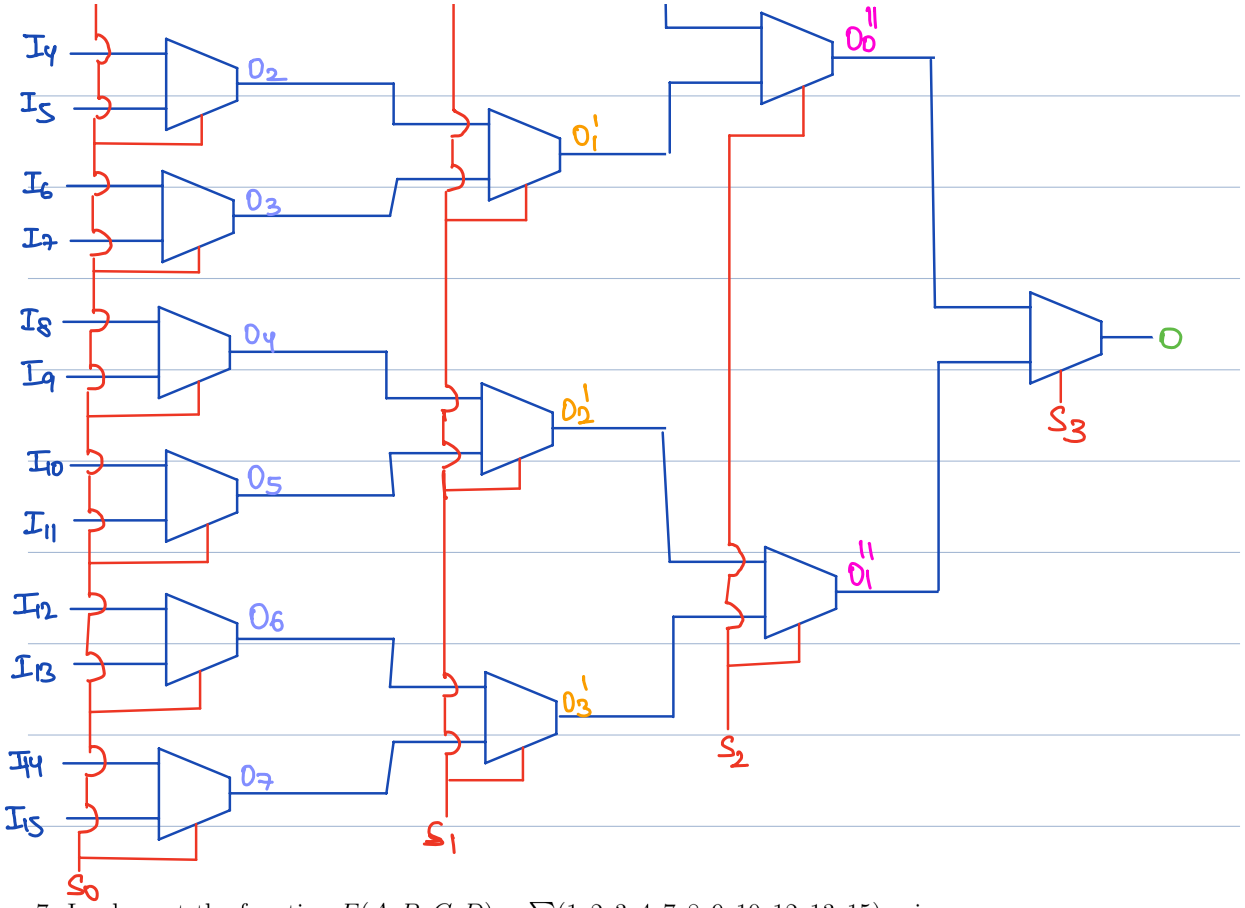
$$F = E.(I_0\bar{A_1}\bar{A_0} + I_1\bar{A_1}A_0 + I_2A_1\bar{A_0} + I_3A_1A_0)$$

$$F = \bar{Z}.(XY\bar{Z} + XYZ + Y\bar{Y}\bar{Z} + \bar{Y}.\bar{Y}Z)$$

$$F = XY\bar{Z}$$

4. Realize a 16-to-1 MUX using only 2-to-1 MUXes. Draw the circuit.





7. Implement the function $F(A, B, C, D) = \sum(1, 2, 3, 4, 7, 8, 9, 10, 12, 13, 15)$ using:

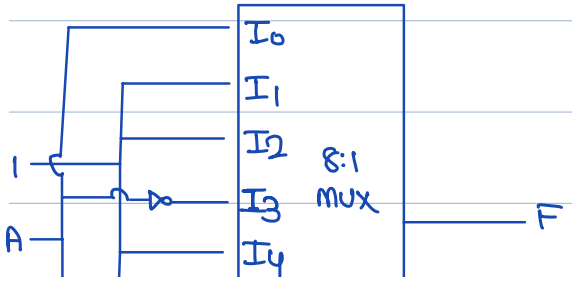
   (a) An 8:1 MUX and required logic gates.

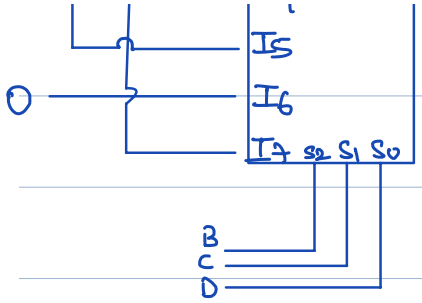
   (b) A 4:1 MUX and required logic gates.

Draw the implementation tables for both the cases.

(a)

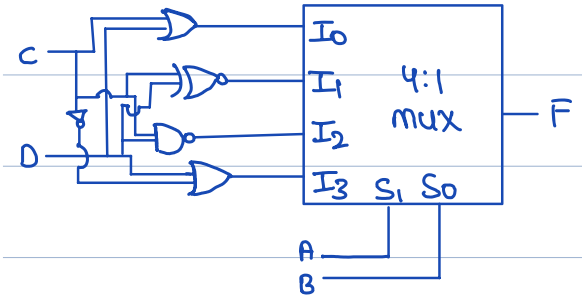
	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
A'	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	A	1	1	A'	1	A	0	1



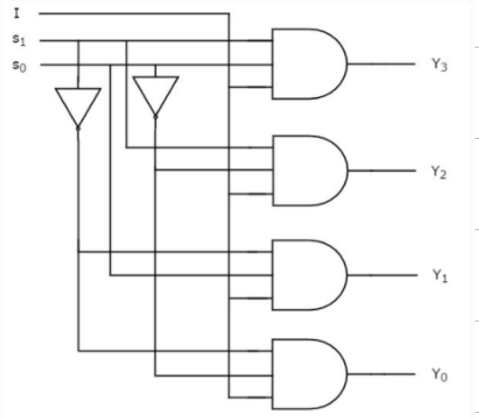


(b)

AB / S_1S_0	I_0 00	I_1 01	I_2 10	I_3 11
$\bar{C}\bar{D}$ (00)	0	4	8	12
$\bar{C}D$ (01)	1	5	9	13
$C\bar{D}$ (10)	2	6	10	14
CD (11)	3	7	11	15
	C+D	C⊙D	$\bar{C}\bar{D}$	$\bar{C}+D$



8. Construct the truth table and output expressions Y_0, Y_1, Y_2, Y_3 for the combinational circuit in Figure 2 with input I and s_1 (MSB), s_0 as the select lines. Comment on its operation by comparing it with a multiplexer.



I	S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0

1	0	0	1	0	0	0	$Y_0 = I\bar{S}_1\bar{S}_0$
1	0	1	0	1	0	0	$Y_1 = I\bar{S}_1 S_0$
1	1	0	0	0	1	0	$Y_2 = IS_1\bar{S}_0$
1	1	1	0	0	0	1	$Y_3 = IS_1S_0$

Depending on values of S_1 and S_0, input I is directed to one of the outputs Y_0, Y_1, Y_2 and Y_3 while the other outputs remain 0. It's like a single input 4 output demultiplexer.

1. Design an Excess-3 to Binary converter using AND gates, OR gates and inverters. Develop the appropriate truth tables and Boolean equations (for the outputs) in terms of minterms and don't cares (take unused cases as don't cares). Perform the appropriate simplification and draw the circuit.

Note: Excess-3 (XS-3) code is a self-complementary binary-coded decimal (BCD) system in which each decimal digit (0-9) is represented by its binary equivalent plus 3. It is useful in error detection. (Since it is self-complementary, if a digit is incorrectly represented, the resulting code may not represent a valid Excess-3 representation.)

A	B	C	D	W	X	Y	Z
0	0	0	0	X	X	X	X
0	0	0	1	X	X	X	X
0	0	1	0	X	X	X	X
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0

0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

W:

AB \ CD	00	01	11	10
00	X	X		X
01				
11	1	X	X	X
10			1	

$W = AB + ACD$

X :

AB \ CD	00	01	11	10
00	X	X		X
01			1	
11		X	X	X
10	1	1		1

$X = BCD + B'C' + B'D'$

Y :

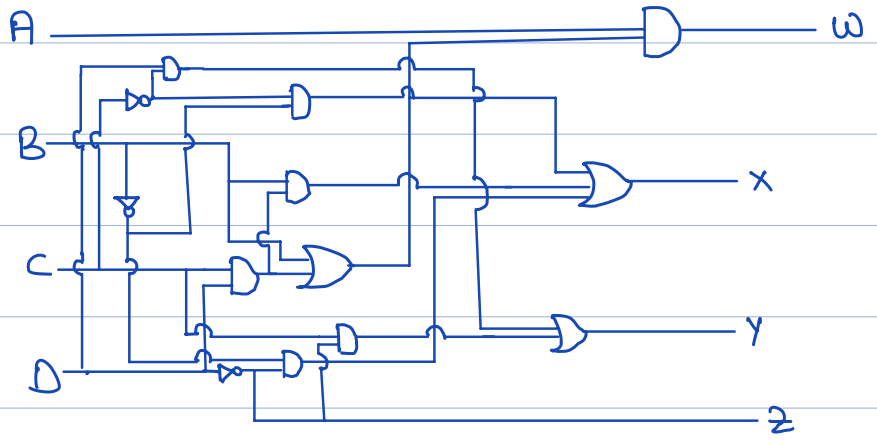
AB \ CD	00	01	11	10
00	X	X		X
01		1		1
11		X	X	X
10		1		1

$Y = C'D + CD'$

Z :

AB \ CD	00	01	11	10
00	X	X		X
01	1			1
11	1	X	X	X
10	1			1

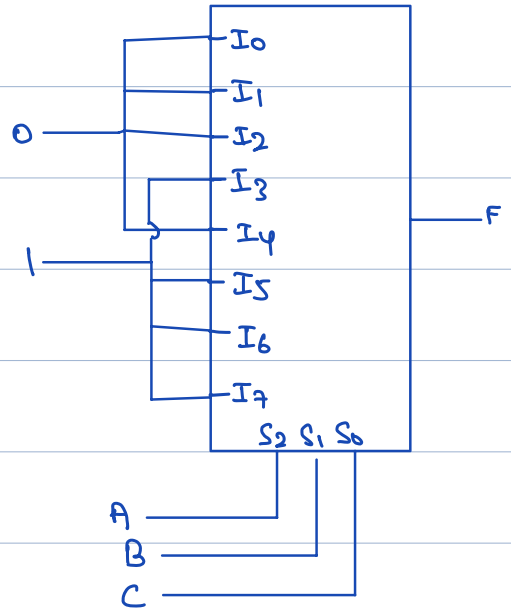
$Z = D'$



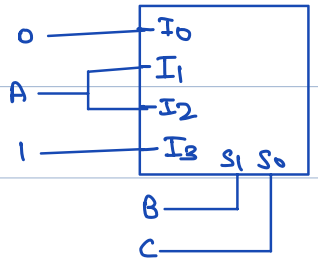
5. Consider the 3-input majority function. Design the logic circuit for this using (a) 8:1 Mux and (b) 4:1 Mux. Choose A(MSB),B and C as inputs.

Majority function: $\Sigma m(3,5,6,7)$

(a)



(b)

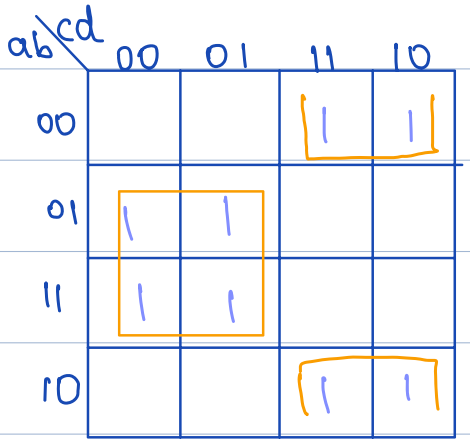


	I_0	I_1	I_2	I_3
A'	0	2	4	6
A	1	3	5	7
	0	A	A	1

6. We have already seen the role of Gray code in Karnaugh maps. In this question, we will look at a conversion task involving 4-bit binary and Gray codes. Consider Table 1 where the input (binary) is given by $abcd$ while the corresponding output, denoted by $g_3g_2g_1g_0$, is in Gray code format. Implement the output g_1 using NOR gates and outputs g_2 and g_0 using NAND gates.

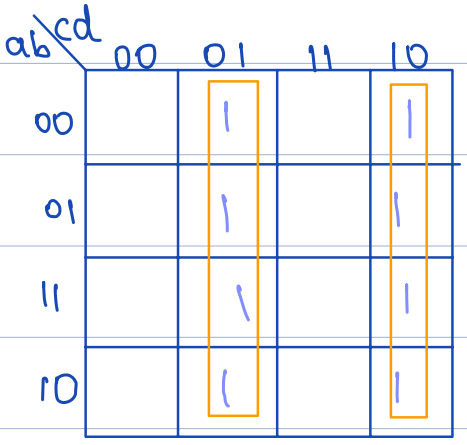
a	b	c	d	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

g_1:



$g_1 = bc' + b'c$

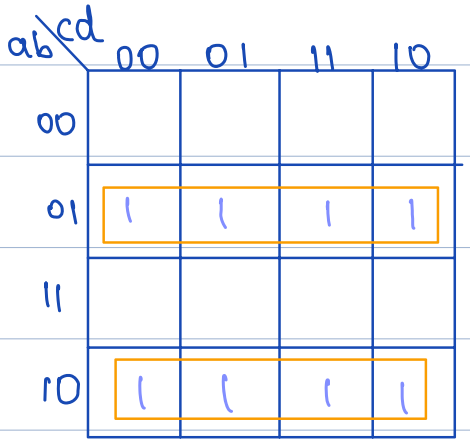
g_0:



$g_0 = c'd + cd'$

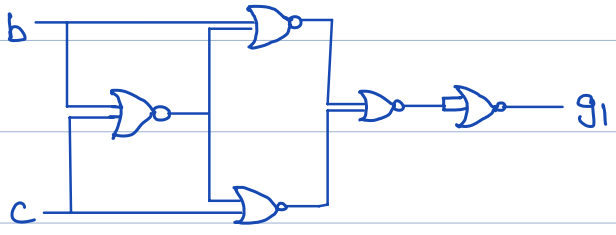
$g_3 = a$

g_2:

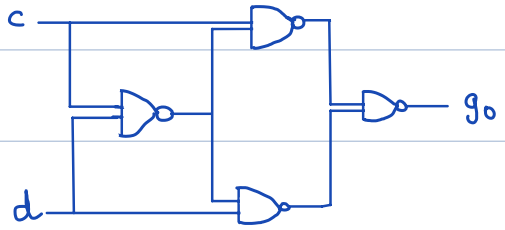


$g_2 = a'b + ab'$

g_1:



g_0:



g_2:

